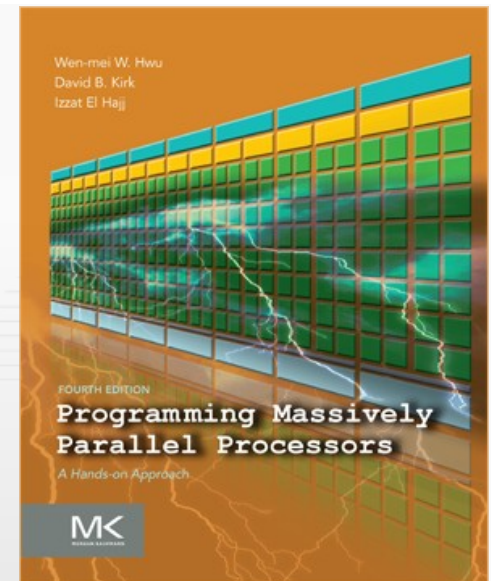


Programming Massively Parallel Processors

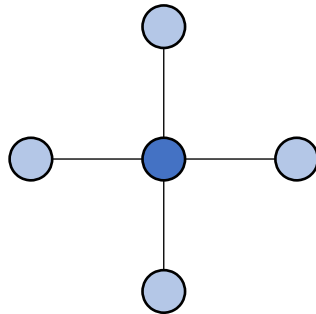
A Hands-on Approach

CHAPTER 8 > Stencil

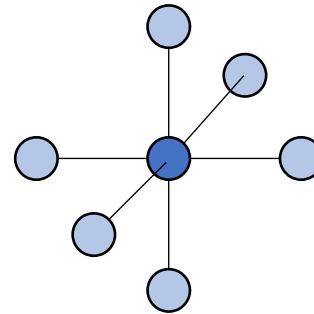


- The **stencil** computation pattern refers to a class of computations on a grid where the value at a grid point is computed based on neighboring points
 - Typically used in solving partial differential equations in domains such as fluid dynamics, heat conductance, combustion, weather forecasting, etc.

- Example:

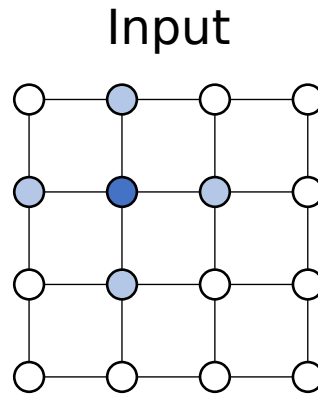


5-point stencil (2D)

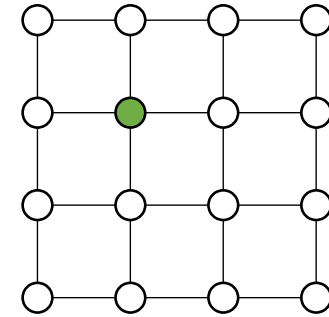


7-point stencil (3D)

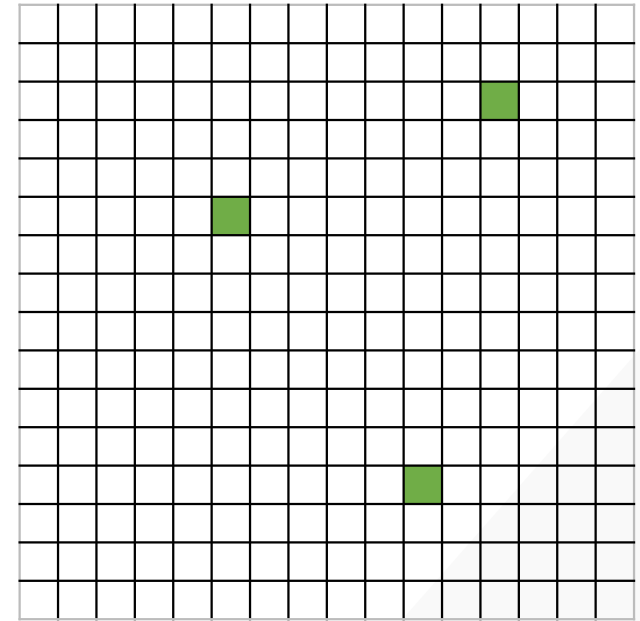
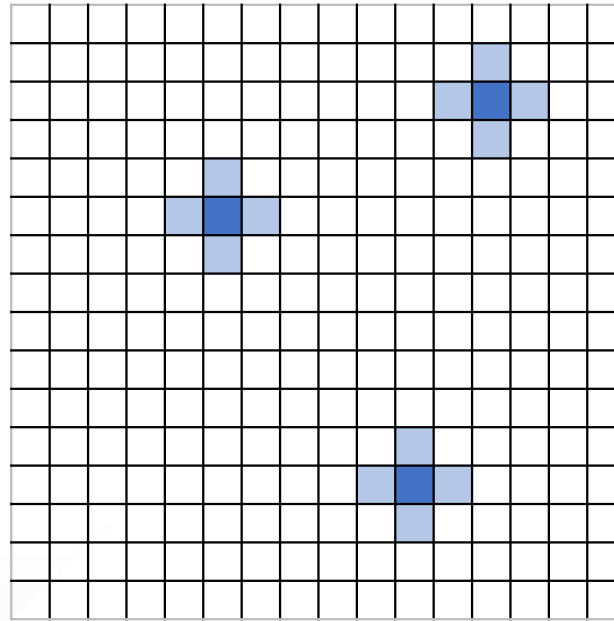
Grid of points:
(4x4 grid shown)



Output

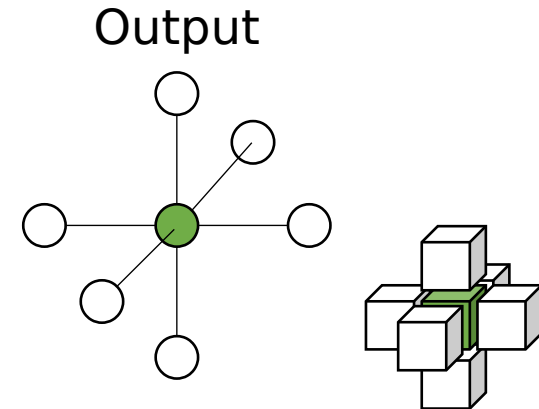
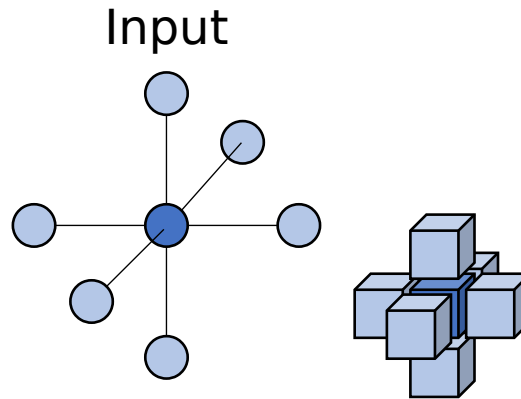


Stored as 2D array:
(16x16 grid shown)

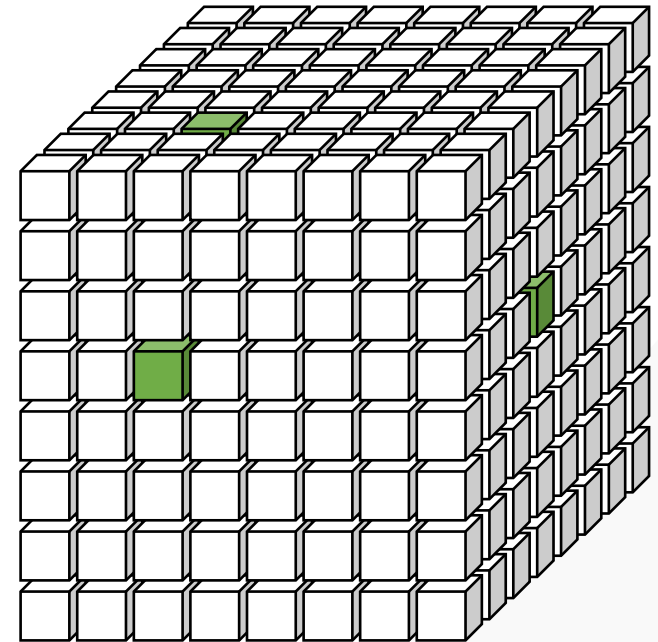
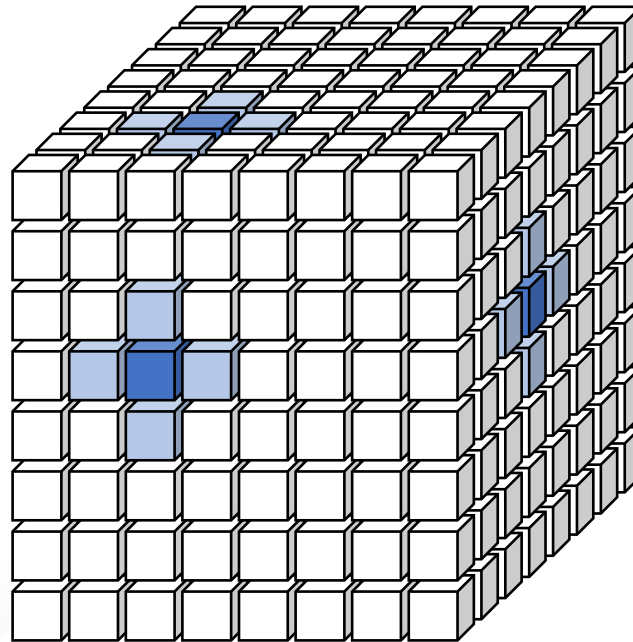


put value is computed based on the **corresponding and neighboring input**

Grid of points:
(one stencil shown)

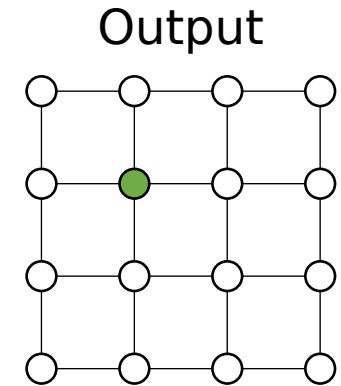
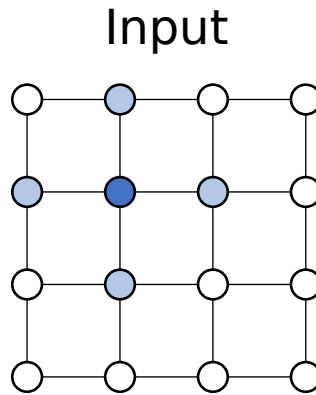


Stored as 3D array:
(8x8x8 grid shown)

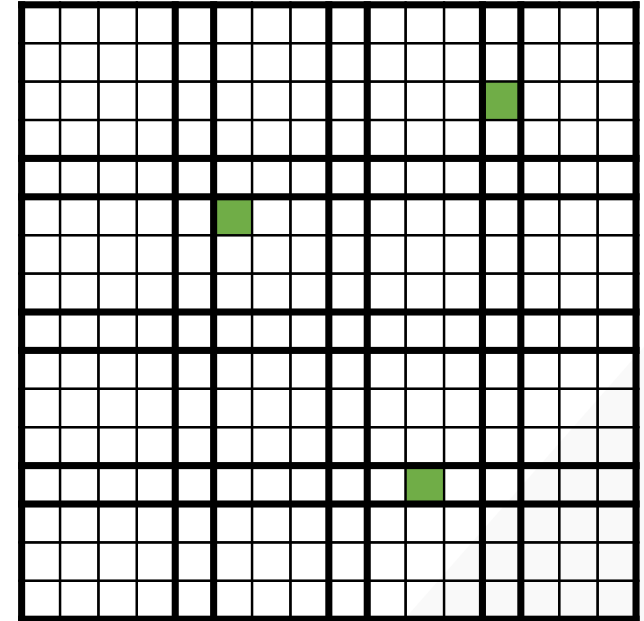
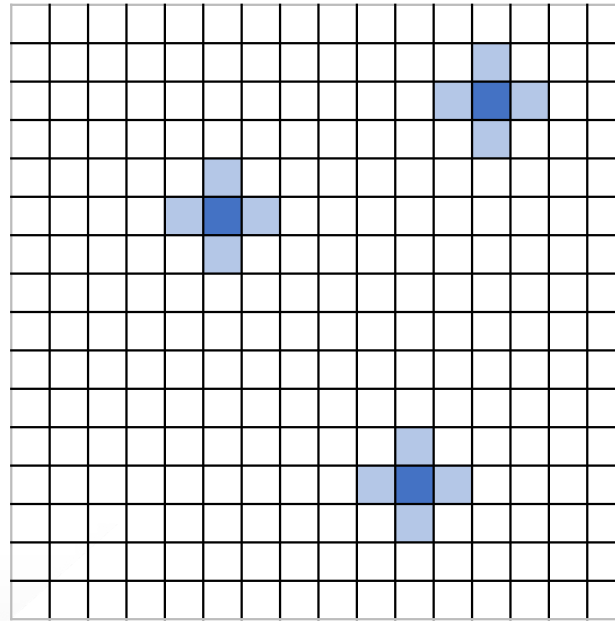


put value is computed based on the **corresponding and neighboring input**

Grid of points:
(4x4 grid shown)



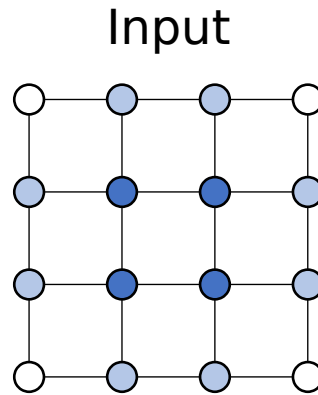
Stored as 2D array:
(16x16 grid shown)



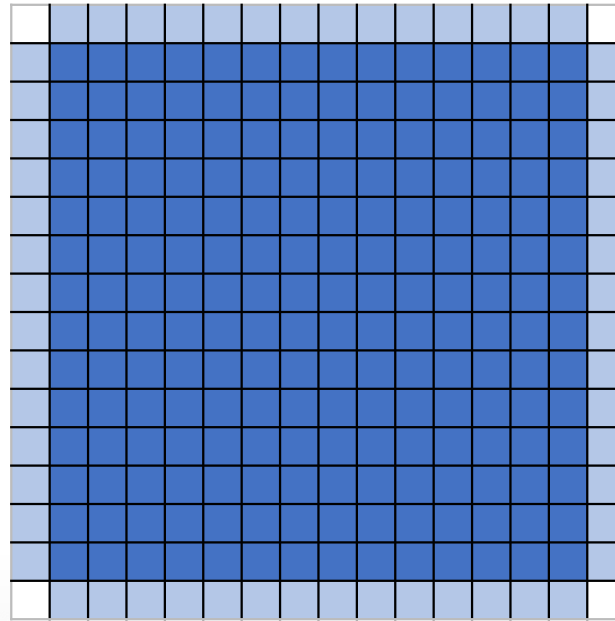
Parallelization Approach:

Assign one thread
per output grid point
(use 2D grid of
threads)

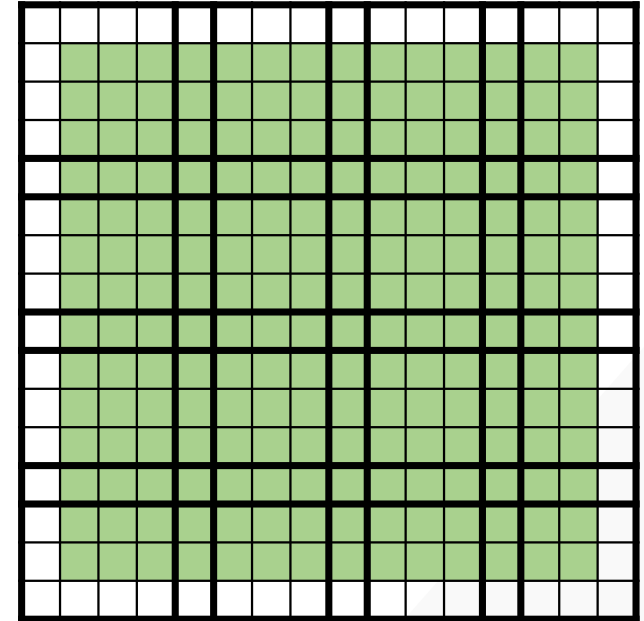
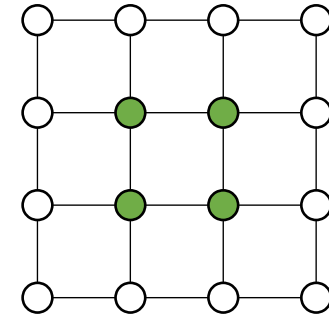
Grid of points:
(4x4 grid shown)



Stored as 2D array:
(16x16 grid shown)

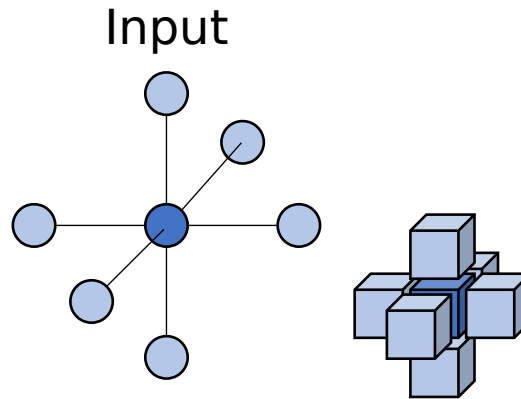


Output

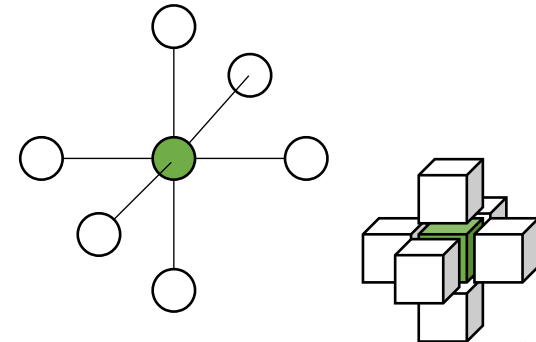


Only compute internal output values such that all input values are in bounds
(input values at the boundary typically store boundary conditions)

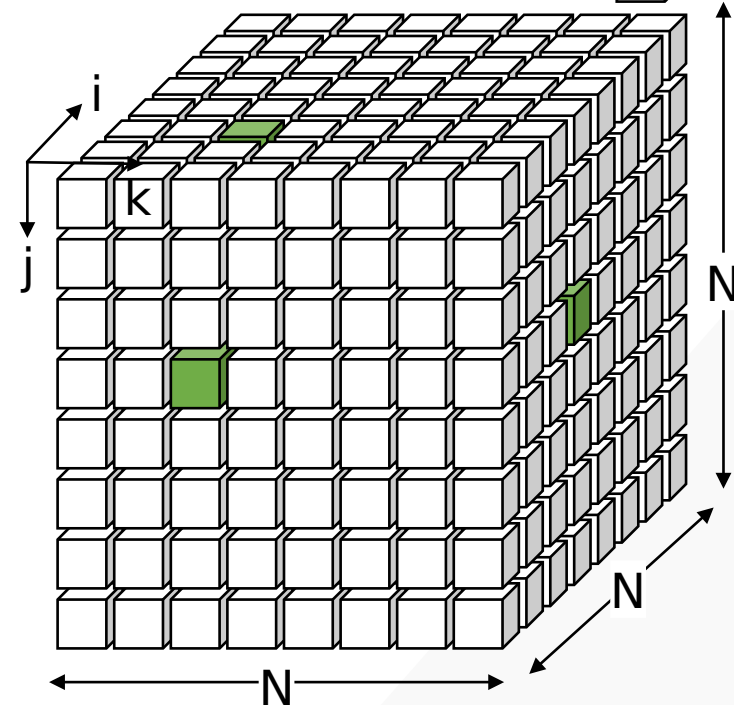
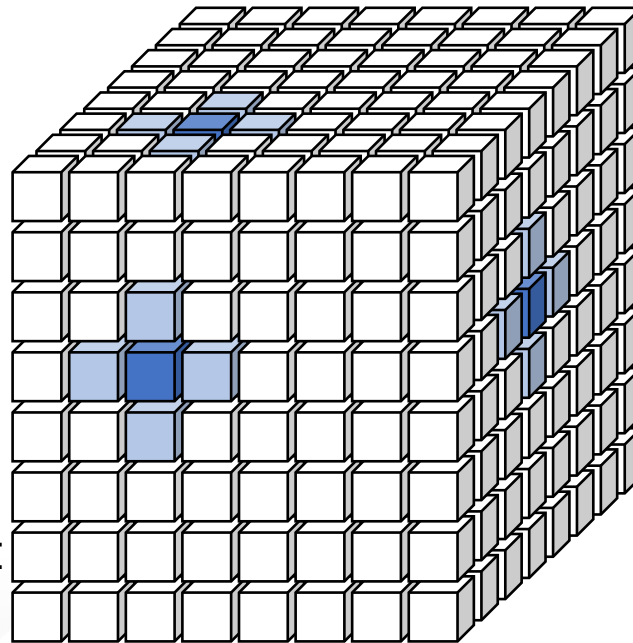
Grid of points:
(one stencil shown)



Output



Stored as 3D array:
(8x8x8 grid shown)

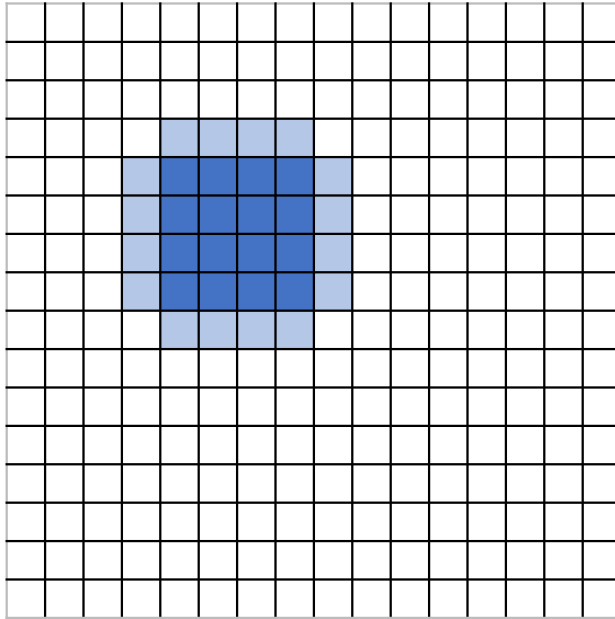


Parallelization Approach:

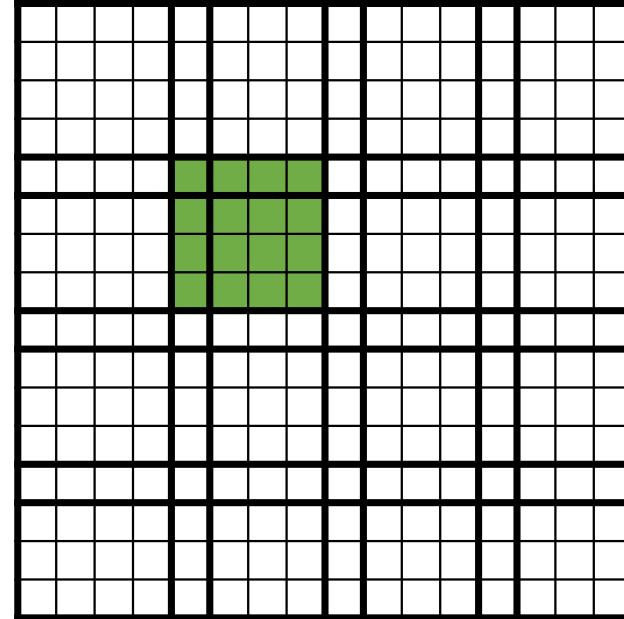
Assign one thread
per output grid point
(use 3D grid of
threads)

```
#define BLOCK_DIM 8
```

```
__global__ void stencil_kernel(float* in, float* out, unsigned int N) {  
    unsigned int i = blockIdx.z*blockDim.z + threadIdx.z;  
    unsigned int j = blockIdx.y*blockDim.y + threadIdx.y;  
    unsigned int k = blockIdx.x*blockDim.x + threadIdx.x;  
    if(i >= 1 && i < N - 1 && j >= 1 && j < N - 1 && k >= 1 && k < N - 1) {  
        out[i*N*N + j*N + k] = C0*in[i*N*N + j*N + k]  
            + C1*in[i*N*N + j*N + (k - 1)]  
            + C2*in[i*N*N + j*N + (k + 1)]  
            + C3*in[i*N*N + (j - 1)*N + k]  
            + C4*in[i*N*N + (j + 1)*N + k]  
            + C5*in[(i - 1)*N*N + j*N + k]  
            + C6*in[(i + 1)*N*N + j*N + k];  
    }  
}
```

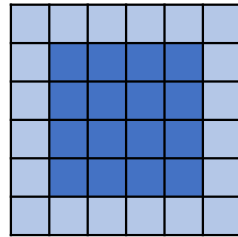



input
(in global
memory)

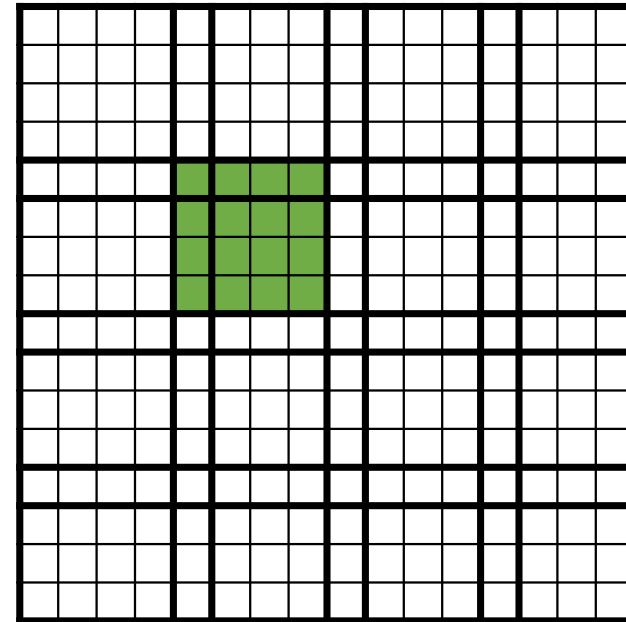


output

Observation: Threads in the same block load some of the same input elements



input_{tile}
(in shared
memory)

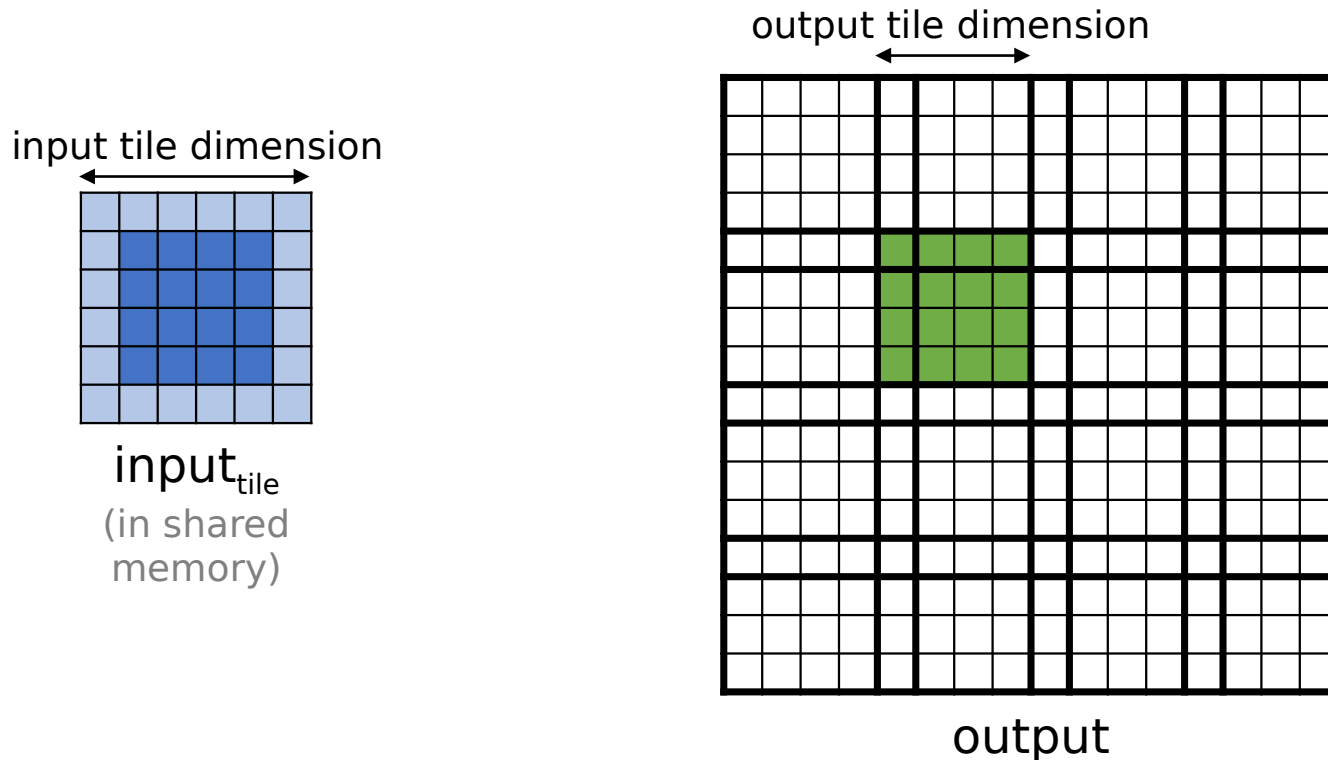


output

Observation: Threads in the same block load some of the same input elements

Optimization: Each thread loads one input element to shared memory and other threads access the element from shared

memory

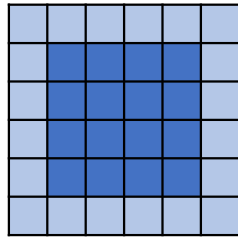


Challenge: Input and output tiles have different dimensions

(output tile dimension = input tile dimension - 2)

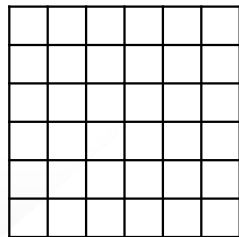
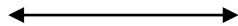
Solution: Launch enough threads per block to load the input tile to shared memory, then use a subset of them to compute and store the output tile

input tile dimension



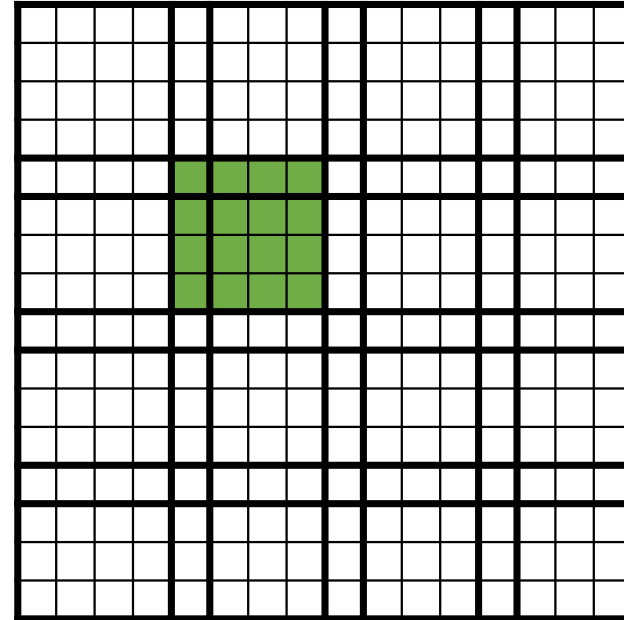
$\text{input}_{\text{tile}}$
(in shared
memory)

input tile dimension



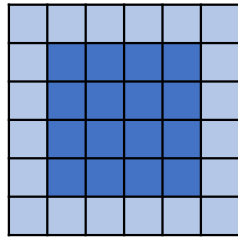
thread
block

output tile dimension



output

input tile dimension

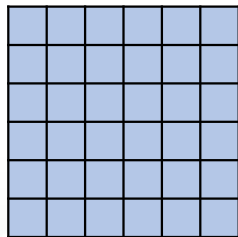


$\text{input}_{\text{tile}}$
(in shared
memory)

input tile dimension

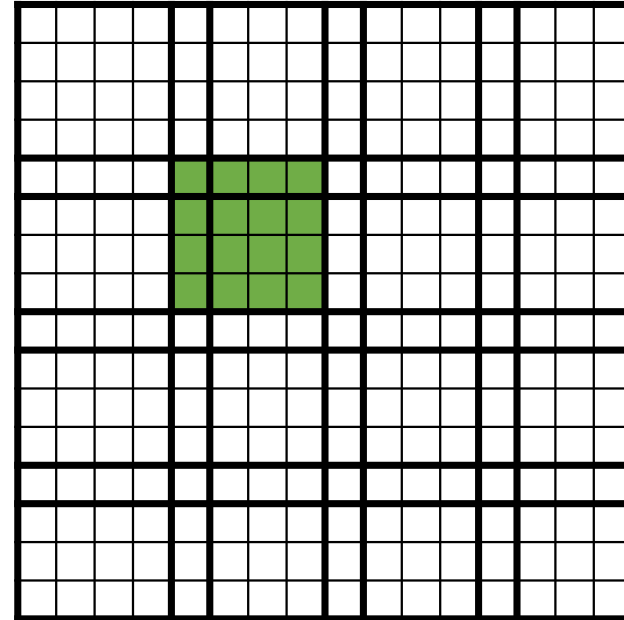


**all threads
active when
loading the
input tile**



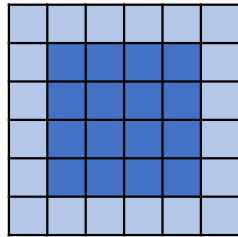
thread
block

output tile dimension



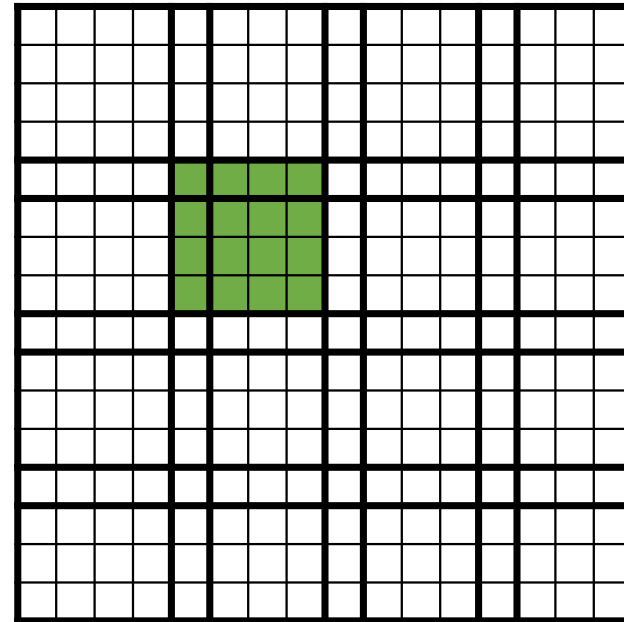
output

input tile dimension



input_{tile}
(in shared
memory)

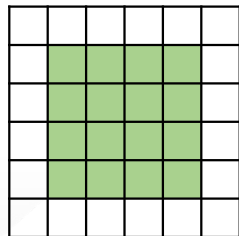
output tile dimension



output

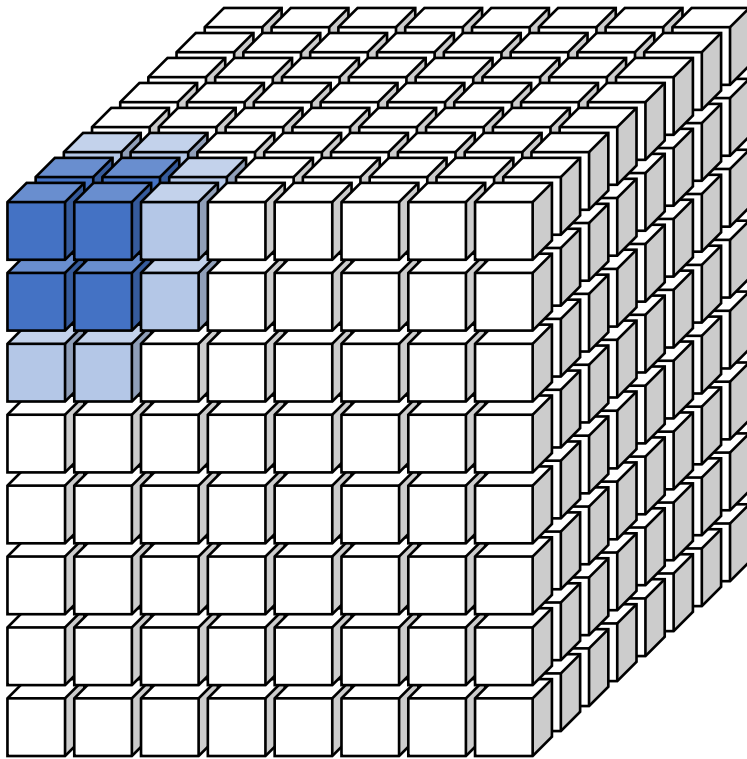
input tile dimension

output tile dimension

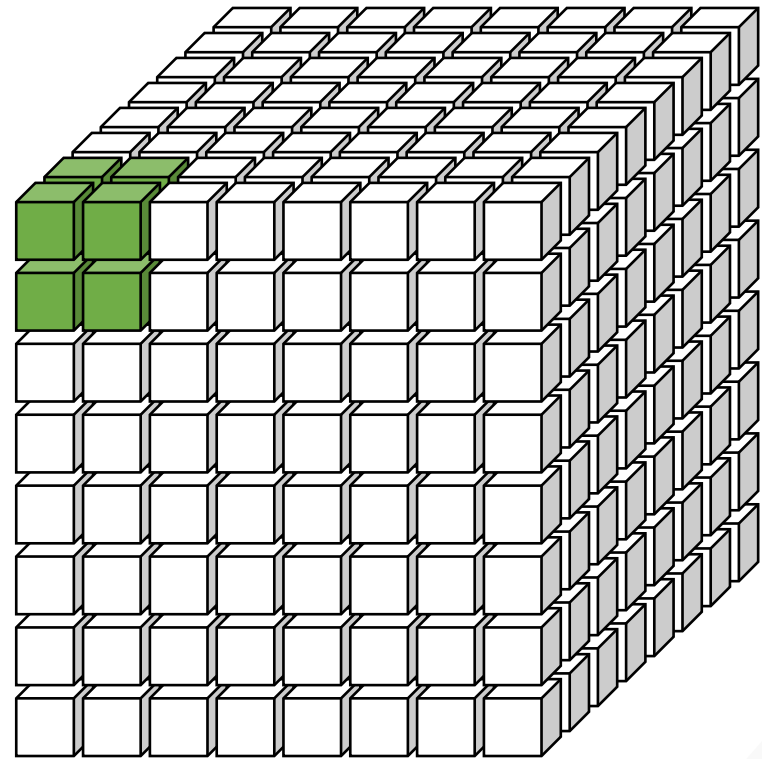


thread
block

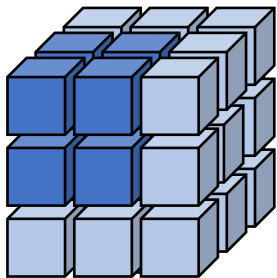
**only internal
threads active
when
computing and
storing the
output tile**



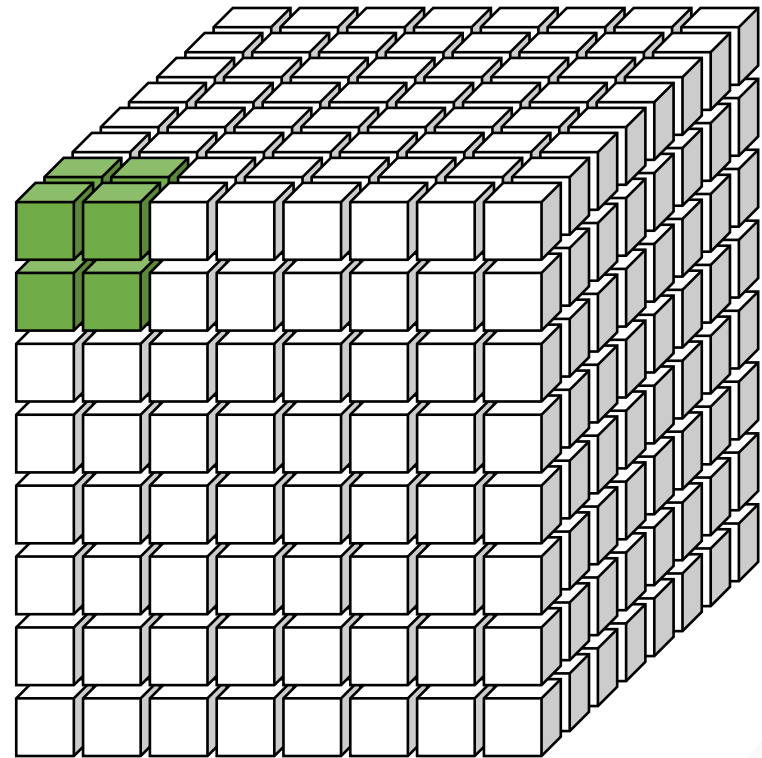
input
(in global
memory)



output



$\text{input}_{\text{tile}}$
(in shared
memory)



output


```
#define BLOCK_DIM 8
#define IN_TILE_DIM BLOCK_DIM
#define OUT_TILE_DIM (IN_TILE_DIM - 2)

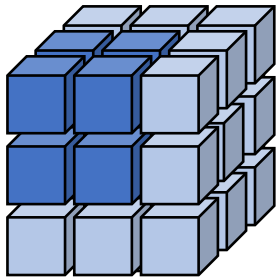
__global__ void stencil_kernel(float* in, float* out, unsigned int N) {

    int i = blockIdx.z*OUT_TILE_DIM + threadIdx.z - 1;
    int j = blockIdx.y*OUT_TILE_DIM + threadIdx.y - 1;
    int k = blockIdx.x*OUT_TILE_DIM + threadIdx.x - 1;

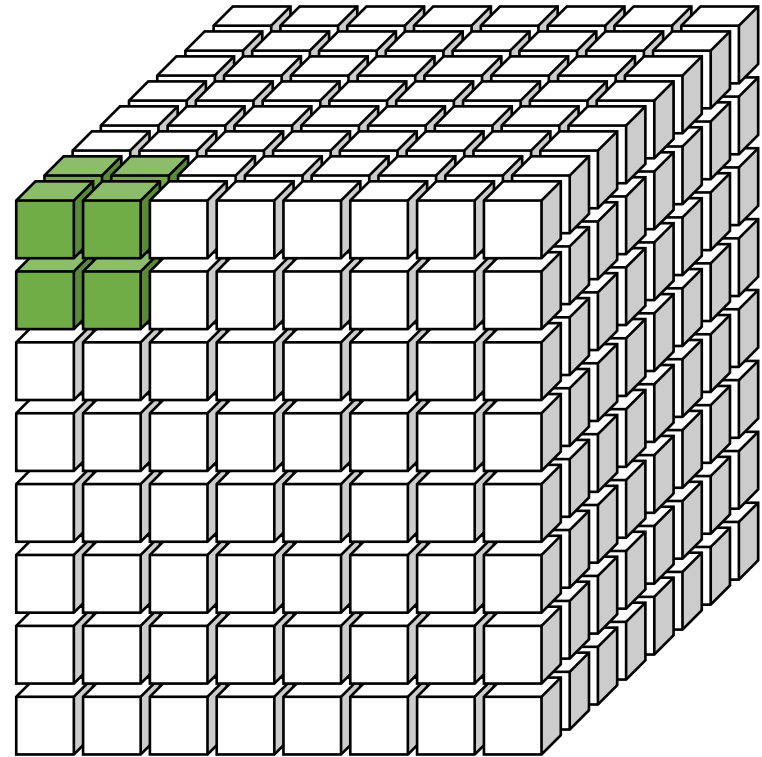
    __shared__ float in_s[IN_TILE_DIM][IN_TILE_DIM][IN_TILE_DIM];
    if(i >= 0 && i < N && j >= 0 && j < N && k >= 0 && k < N) {
        in_s[threadIdx.z][threadIdx.y][threadIdx.x] = in[i*N*N + j*N + k];
    }
    __syncthreads();

    if(i >= 1 && i < N - 1 && j >= 1 && j < N - 1 && k >= 1 && k < N - 1) {
        if(threadIdx.z >= 1 && threadIdx.z < IN_TILE_DIM - 1 && threadIdx.y >= 1
            && threadIdx.y < IN_TILE_DIM - 1 && threadIdx.x >= 1 && threadIdx.x < IN_TILE_DIM - 1) {
            out[i*N*N + j*N + k] = C0*in_s[threadIdx.z][threadIdx.y][threadIdx.x]
                + C1*in_s[threadIdx.z][threadIdx.y][threadIdx.x - 1]
                + C2*in_s[threadIdx.z][threadIdx.y][threadIdx.x + 1]
                + C3*in_s[threadIdx.z][threadIdx.y - 1][threadIdx.x]
                + C4*in_s[threadIdx.z][threadIdx.y + 1][threadIdx.x]
                + C5*in_s[threadIdx.z - 1][threadIdx.y][threadIdx.x]
                + C6*in_s[threadIdx.z + 1][threadIdx.y][threadIdx.x];
        }
    }
}
```

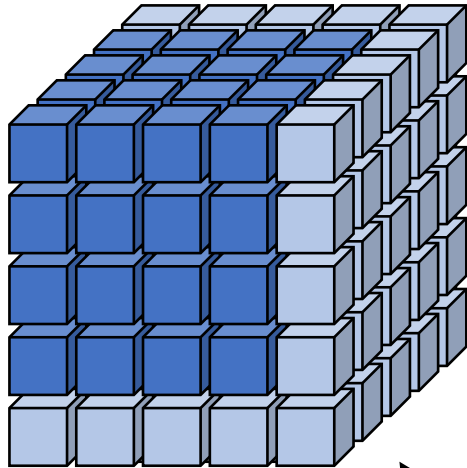
- Original kernel:
 - Each thread performs 13 OPs (6 FP adds and 7 FP muls)
 - Each thread loads 28 B from global memory (7 FP values)
 - Ratio: $(13 \text{ OPs}) / (28 \text{ B}) = 0.46 \text{ OP/B}$
- Tiled kernel:
 - Assume the input tile size is T (output tile size is $T - 2$)
 - Each block performs $(13 \text{ OPs}) * (T - 2)^3 = 13(T - 2)^3 \text{ OPs}$
 - Each block loads $(4 \text{ B}) * T^3$
 - Ratio: $[13 * (T - 2)^3] / [4 * T^3] = 3.25 * (1 - 2/T)^3$
 - For $T=8$, the ratio is 1.37 OP/B
 - Increasing T will improve the ratio
 - For $T=32$, ratio is 2.68 ($\approx 2\times$ improvement)
 - Intuition: boundary elements have lower data reuse, and increasing tile size decreases the ratio of boundary elements to total elements



$\text{input}_{\text{tile}}$
(in shared
memory)

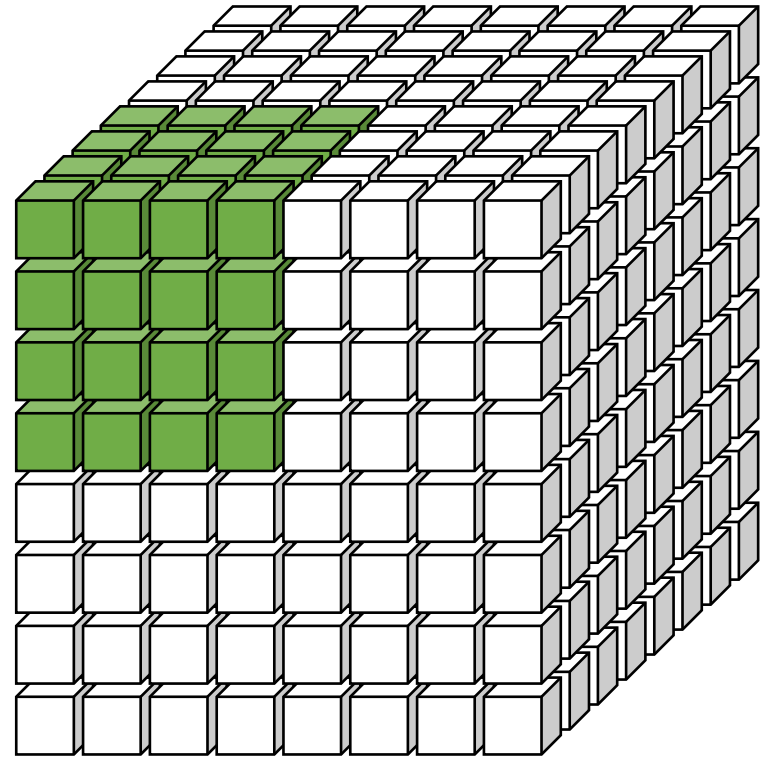


output



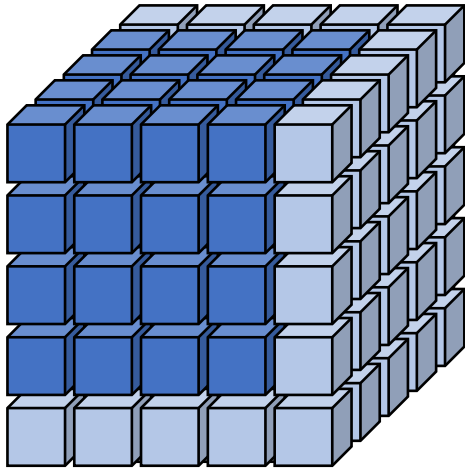
$\text{input}_{\text{tile}}$
(in shared
memory)

Fewer boundary
elements
relative to
internal
elements



output

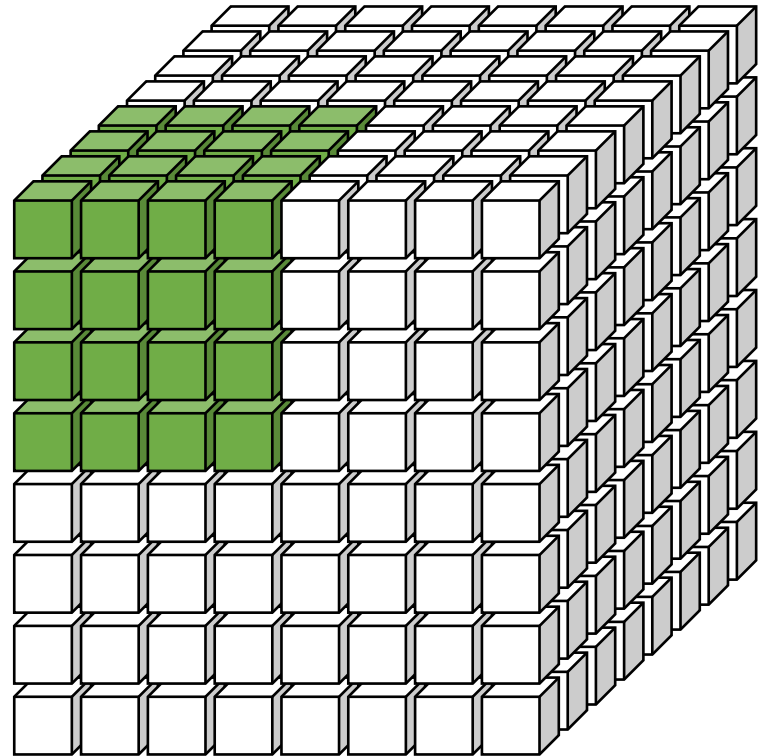
- Challenges with increasing input tile size:
 - Block size limit: The input tile size in the current implementation is the same as the block size which is limited by the hardware
 - Shared memory capacity limit: The input tile size in the current implementation determines the shared memory usage per block which is limited by hardware
 - Even if the limit is not exceeded, using too much shared memory may hurt occupancy
- Solutions:
 - Block size limit: Use thread coarsening to process a larger input/output tile without using more threads
 - Price of parallelization here is redundant loading of boundary elements
 - Shared memory capacity limit: Only keep needed slices of the input tile in shared memory



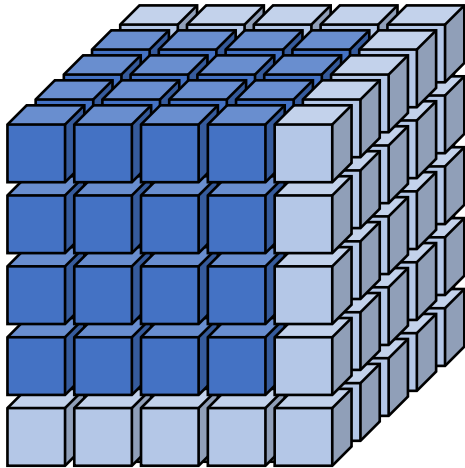
input_{tile}
(in shared
memory)

Challenge:
Input tile requires
too much shared
memory
(hurts occupancy
or even exceeds
limit)

Challenge:
Maximum
threads per
block exceeded



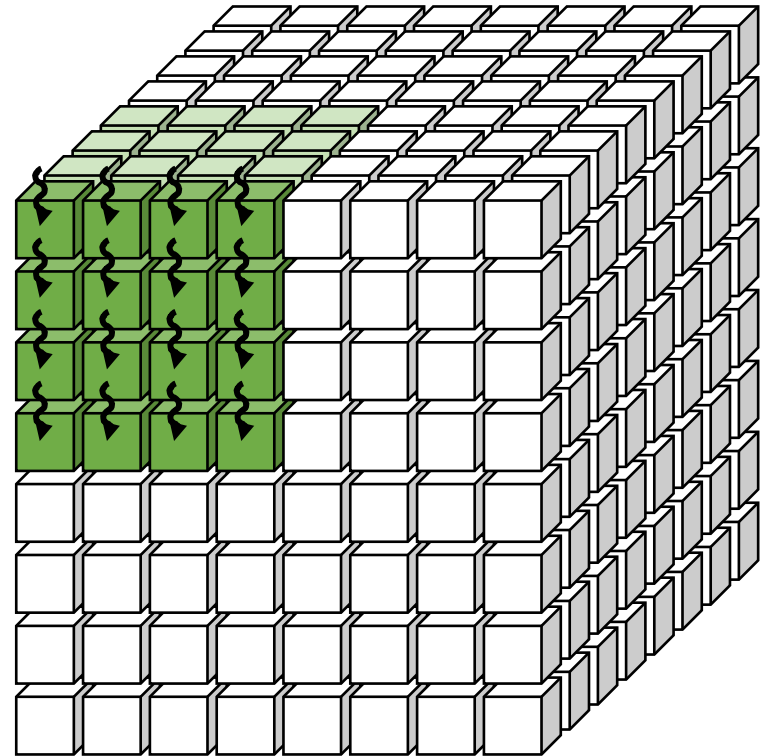
output



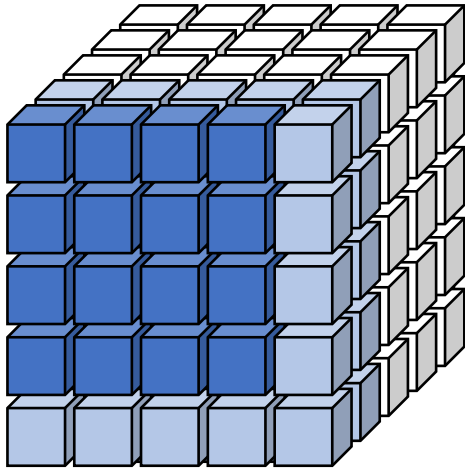
input_{tile}
(in shared
memory)

Challenge:
Input tile requires
too much shared
memory
(hurts occupancy
or even exceeds
limit)

Solution:
Assign enough
threads for
loading one
input plane and
processing one
output plane



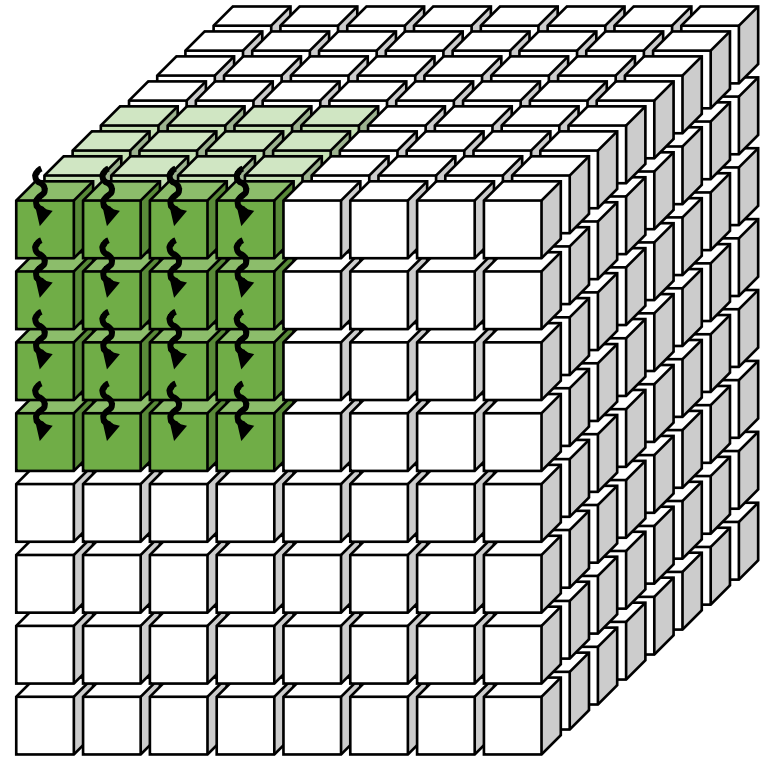
output



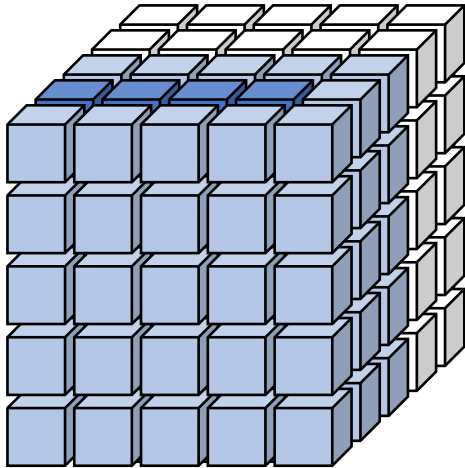
input_{tile}
(in shared
memory)

Solution:
Assign enough
threads for
loading one
input plane and
processing one
output plane

Solution:
Only store the three
input planes needed
by the output plane
at a time



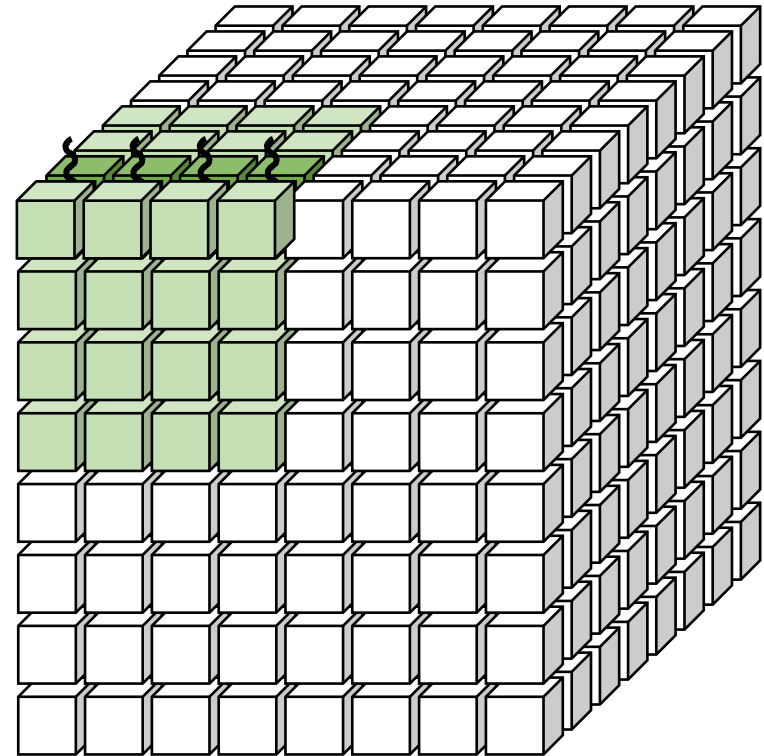
output



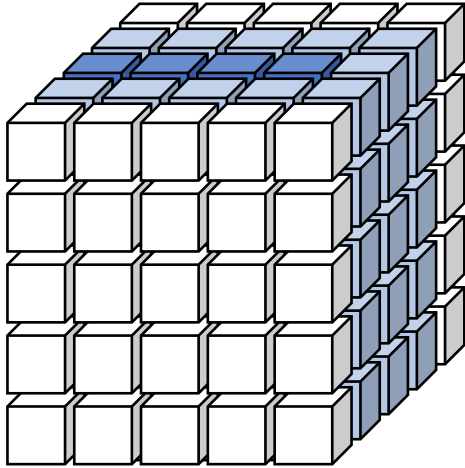
input_{tile}
(in shared
memory)

Solution:
Assign enough
threads for
loading one
input plane and
processing one
output plane

Solution:
Only store the three
input planes needed
by the output plane
at a time



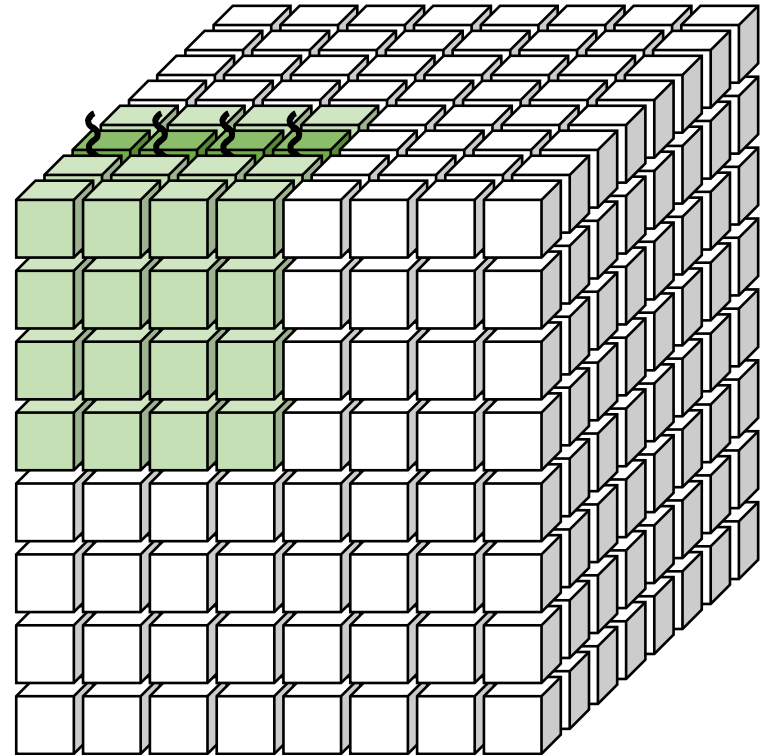
output



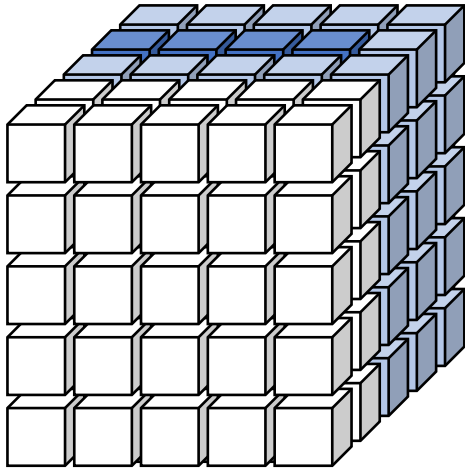
input_{tile}
(in shared
memory)

Solution:
Assign enough
threads for
loading one
input plane and
processing one
output plane

Solution:
Only store the three
input planes needed
by the output plane
at a time



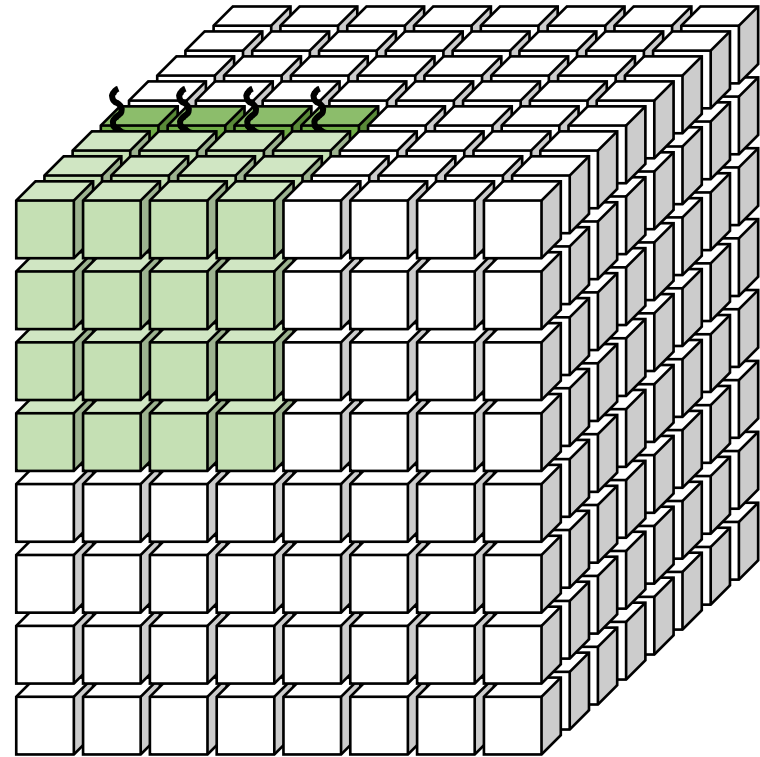
output



input_{tile}
(in shared
memory)

Solution:
Assign enough
threads for
loading one
input plane and
processing one
output plane

Solution:
Only store the three
input planes needed
by the output plane
at a time



output

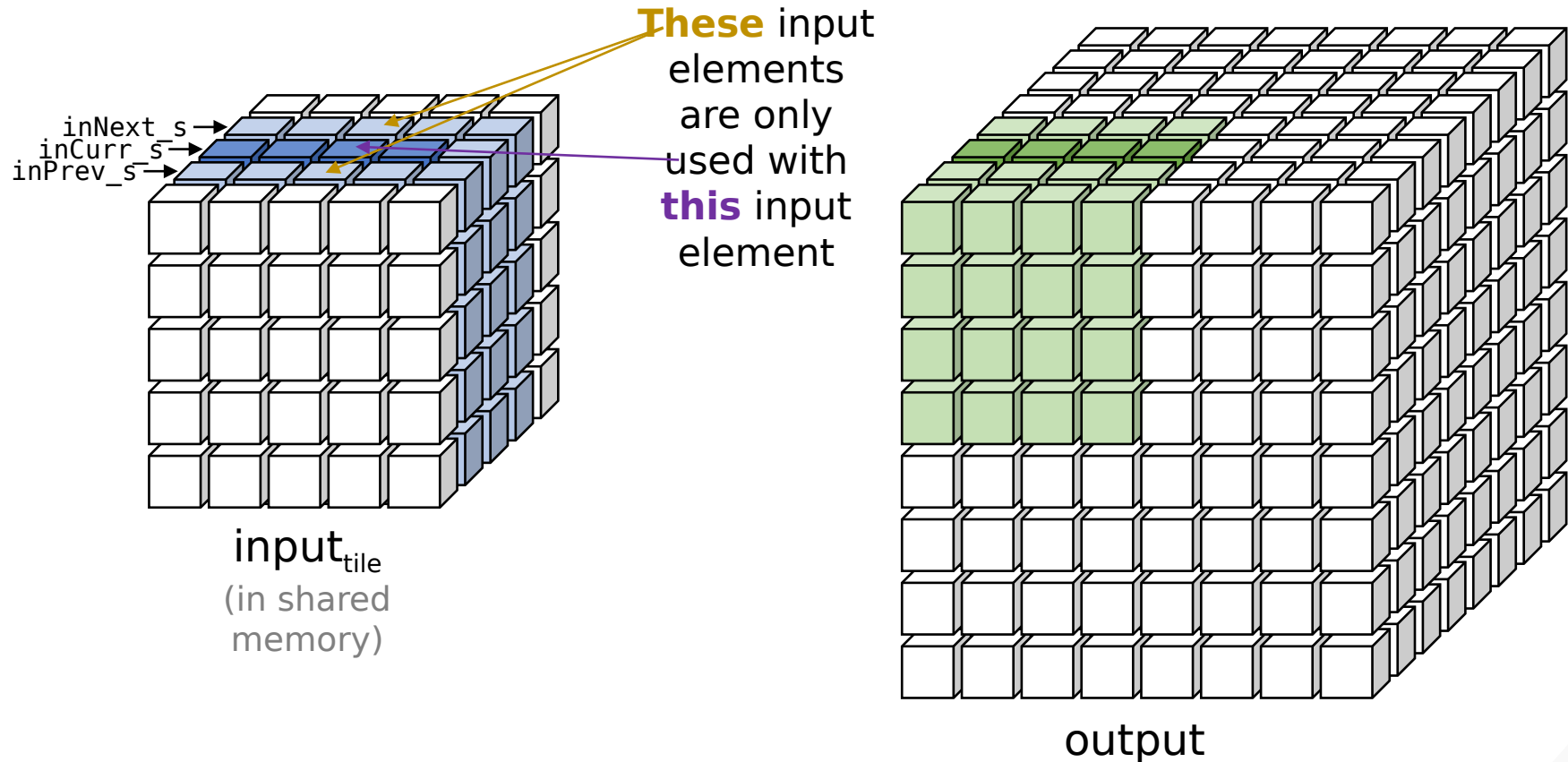
Stencil with Thread Coarsening Code

```
__global__ void stencil_kernel(float* in, float* out, unsigned int N) {

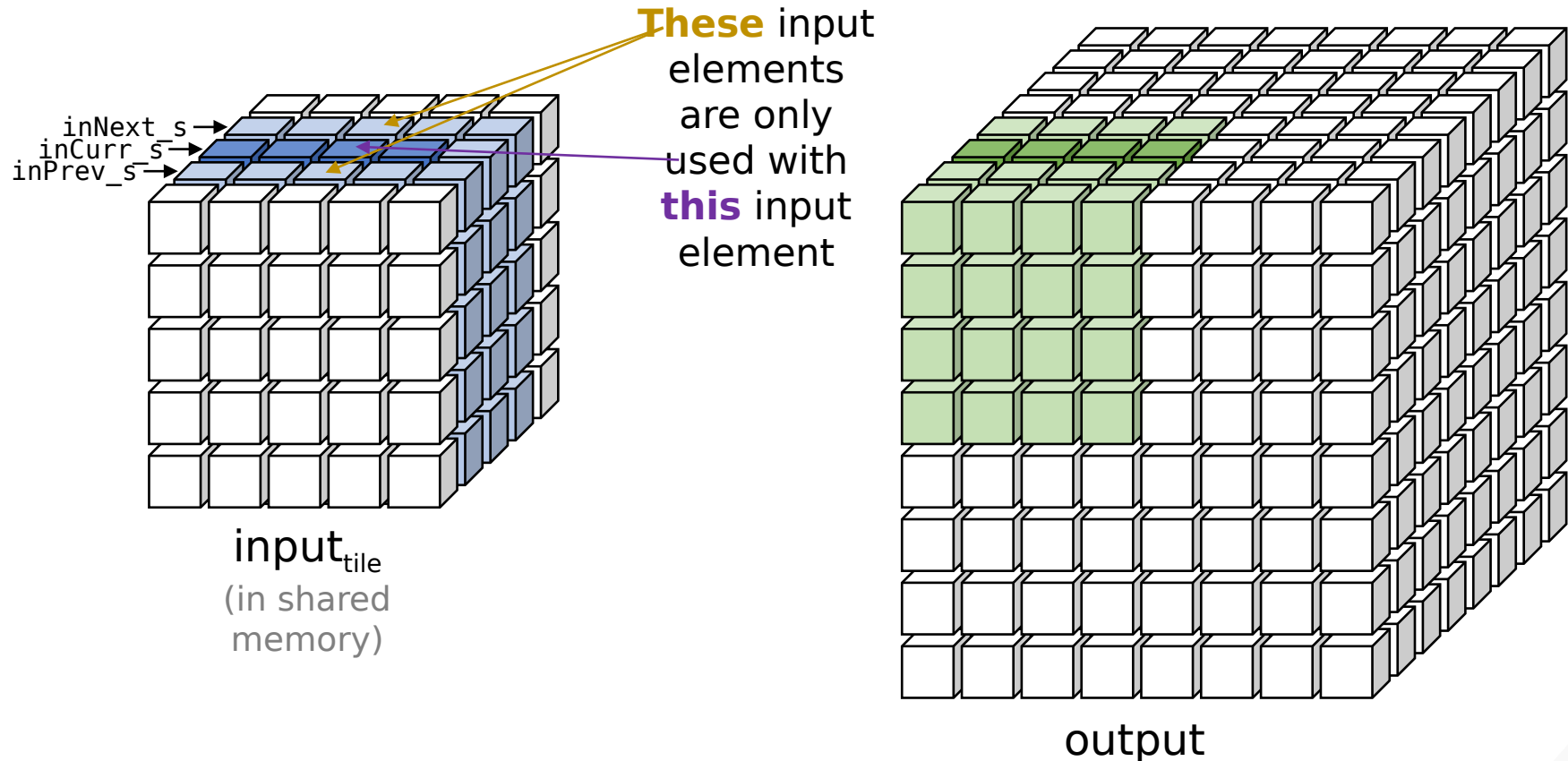
    int iStart = blockIdx.z*OUT_TILE_DIM;
    int j = blockIdx.y*OUT_TILE_DIM + threadIdx.y - 1;
    int k = blockIdx.x*OUT_TILE_DIM + threadIdx.x - 1;

    __shared__ float inPrev_s[IN_TILE_DIM][IN_TILE_DIM];
    __shared__ float inCurr_s[IN_TILE_DIM][IN_TILE_DIM];
    __shared__ float inNext_s[IN_TILE_DIM][IN_TILE_DIM];
    if(iStart - 1 >= 0 && iStart - 1 < N && j >= 0 && j < N && k >= 0 && k < N) {
        inPrev_s[threadIdx.y][threadIdx.x] = in[(iStart - 1)*N*N + j*N + k];
    }
    if(iStart >= 0 && iStart < N && j >= 0 && j < N && k >= 0 && k < N) {
        inCurr_s[threadIdx.y][threadIdx.x] = in[iStart*N*N + j*N + k];
    }

    for(int i = iStart; i < iStart + OUT_TILE_DIM; ++i) {
        if(i + 1 >= 0 && i + 1 < N && j >= 0 && j < N && k >= 0 && k < N) {
            inNext_s[threadIdx.y][threadIdx.x] = in[(i + 1)*N*N + j*N + k];
        }
        __syncthreads();
        if(i >= 1 && i < N - 1 && j >= 1 && j < N - 1 && k >= 1 && k < N - 1) {
            if(threadIdx.y >= 1 && threadIdx.y < IN_TILE_DIM - 1
               && threadIdx.x >= 1 && threadIdx.x < IN_TILE_DIM - 1) {
                out[i*N*N + j*N + k] = C0*inCurr_s[threadIdx.y][threadIdx.x]
                    + C1*inCurr_s[threadIdx.y][threadIdx.x - 1]
                    + C2*inCurr_s[threadIdx.y][threadIdx.x + 1]
                    + C3*inCurr_s[threadIdx.y - 1][threadIdx.x]
                    + C4*inCurr_s[threadIdx.y + 1][threadIdx.x]
                    + C5*inPrev_s[threadIdx.y][threadIdx.x] +
                    C6*inNext_s[threadIdx.y][threadIdx.x];
            }
        }
        __syncthreads();
        inPrev_s[threadIdx.y][threadIdx.x] = inCurr_s[threadIdx.y][threadIdx.x];
        inCurr_s[threadIdx.y][threadIdx.x] = inNext_s[threadIdx.y][threadIdx.x];
    }
}
```



Observation: Only the current slice is truly shared by the threads. The previous and next slice elements are only needed by the thread that loaded them.



Optimization: Save shared memory by putting the next slice elements in registers, moving them to shared memory when they become the current slice, then moving them back to registers when they become the previous slice. We only need enough shared memory for one slice

The registers of the different threads collectively form a tile (called **register**

Stencil with Thread Coarsening Code

```
__global__ void stencil_kernel(float* in, float* out, unsigned int N) {

    int iStart = blockIdx.z*OUT_TILE_DIM;
    int j = blockIdx.y*OUT_TILE_DIM + threadIdx.y - 1;
    int k = blockIdx.x*OUT_TILE_DIM + threadIdx.x - 1;

    __shared__ float inPrev_s[IN_TILE_DIM][IN_TILE_DIM];
    __shared__ float inCurr_s[IN_TILE_DIM][IN_TILE_DIM];
    __shared__ float inNext_s[IN_TILE_DIM][IN_TILE_DIM];
    if(iStart - 1 >= 0 && iStart - 1 < N && j >= 0 && j < N && k >= 0 && k < N) {
        inPrev_s[threadIdx.y][threadIdx.x] = in[(iStart - 1)*N*N + j*N + k];
    }
    if(iStart >= 0 && iStart < N && j >= 0 && j < N && k >= 0 && k < N) {
        inCurr_s[threadIdx.y][threadIdx.x] = in[iStart*N*N + j*N + k];
    }

    for(int i = iStart; i < iStart + OUT_TILE_DIM; ++i) {
        if(i + 1 >= 0 && i + 1 < N && j >= 0 && j < N && k >= 0 && k < N) {
            inNext_s[threadIdx.y][threadIdx.x] = in[(i + 1)*N*N + j*N + k];
        }
        __syncthreads();
        if(i >= 1 && i < N - 1 && j >= 1 && j < N - 1 && k >= 1 && k < N - 1) {
            if(threadIdx.y >= 1 && threadIdx.y < IN_TILE_DIM - 1
               && threadIdx.x >= 1 && threadIdx.x < IN_TILE_DIM - 1) {
                out[i*N*N + j*N + k] = C0*inCurr_s[threadIdx.y][threadIdx.x]
                    + C1*inCurr_s[threadIdx.y][threadIdx.x - 1]
                    + C2*inCurr_s[threadIdx.y][threadIdx.x + 1]
                    + C3*inCurr_s[threadIdx.y - 1][threadIdx.x]
                    + C4*inCurr_s[threadIdx.y + 1][threadIdx.x]
                    + C5*inPrev_s[threadIdx.y][threadIdx.x]
                    + C6*inNext_s[threadIdx.y][threadIdx.x];
            }
        }
        __syncthreads();
        inPrev_s[threadIdx.y][threadIdx.x] = inCurr_s[threadIdx.y][threadIdx.x];
        inCurr_s[threadIdx.y][threadIdx.x] = inNext_s[threadIdx.y][threadIdx.x];
    }
}
```

Stencil with Register Tiling Code

```
__global__ void stencil_kernel(float* in, float* out, unsigned int N) {

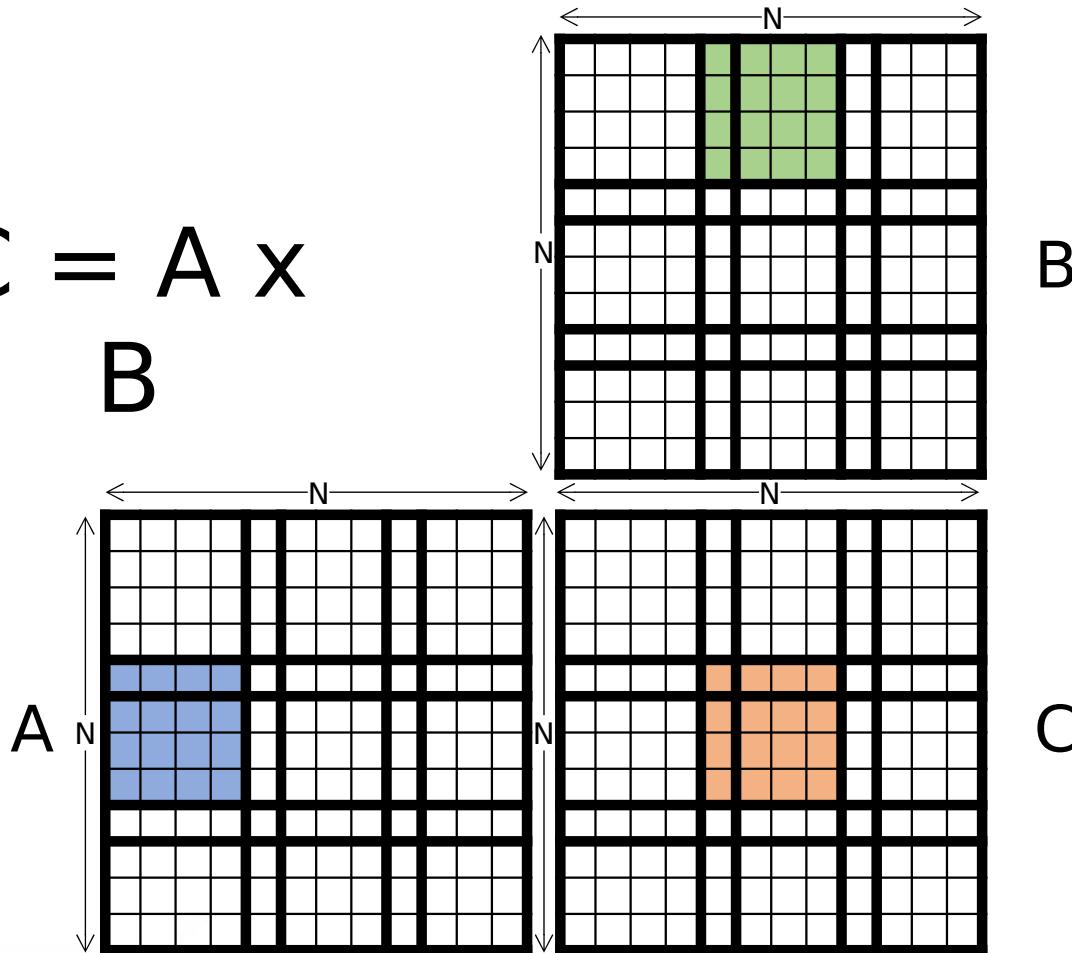
    int iStart = blockIdx.z*OUT_TILE_DIM;
    int j = blockIdx.y*OUT_TILE_DIM + threadIdx.y - 1;
    int k = blockIdx.x*OUT_TILE_DIM + threadIdx.x - 1;

    float inPrev;
    __shared__ float inCurr_s[IN_TILE_DIM][IN_TILE_DIM];
    float inNext;
    if(iStart - 1 >= 0 && iStart - 1 < N && j >= 0 && j < N && k >= 0 && k < N) {
        inPrev = in[(iStart - 1)*N*N + j*N + k];
    }
    if(iStart >= 0 && iStart < N && j >= 0 && j < N && k >= 0 && k < N) {
        inCurr_s[threadIdx.y][threadIdx.x] = in[iStart*N*N + j*N + k];
    }

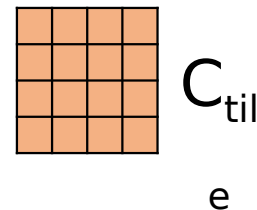
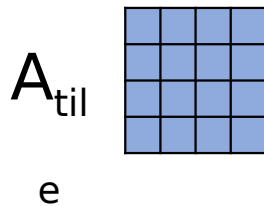
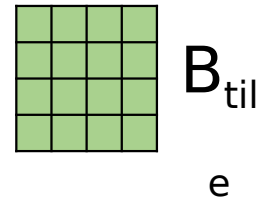
    for(int i = iStart; i < iStart + OUT_TILE_DIM; ++i) {
        if(i + 1 >= 0 && i + 1 < N && j >= 0 && j < N && k >= 0 && k < N) {
            inNext = in[(i + 1)*N*N + j*N + k];
        }
        __syncthreads();
        if(i >= 1 && i < N - 1 && j >= 1 && j < N - 1 && k >= 1 && k < N - 1) {
            if(threadIdx.y >= 1 && threadIdx.y < IN_TILE_DIM - 1
               && threadIdx.x >= 1 && threadIdx.x < IN_TILE_DIM - 1) {
                out[i*N*N + j*N + k] = C0*inCurr_s[threadIdx.y][threadIdx.x]
                    + C1*inCurr_s[threadIdx.y][threadIdx.x - 1]
                    + C2*inCurr_s[threadIdx.y][threadIdx.x + 1]
                    + C3*inCurr_s[threadIdx.y - 1][threadIdx.x]
                    + C4*inCurr_s[threadIdx.y + 1][threadIdx.x]
                    + C5*inPrev
                    + C6*inNext;
            }
        }
        __syncthreads();
        inPrev = inCurr_s[threadIdx.y][threadIdx.x];
        inCurr_s[threadIdx.y][threadIdx.x] = inNext;
    }
}
```


Have we seen register tiling before?

$$C = A \times B$$



$$C_{\text{tile}} = A_{\text{tile}} \times B_{\text{tile}}$$



The A and B input tiles were stored in shared memory.
The C output tile was stored in the registers of the threads collectively.

Stencil just made register tiling more apparent because the same tile was sometimes stored in registers and other

- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.